

# Pro-Sport Complex Management: An Integrated Platform with Escrow-Based Social Matching and Try-Before-You-Buy E-Commerce

Minh Vy Ho Vo<sup>1</sup>, Dat Nguyen Tien Pham<sup>1</sup>, Huy Khang Duong<sup>1</sup>, Phat Tan Ho<sup>1</sup>, and Phuc Dang Nguyen<sup>1</sup>

FPT University, Da Nang, Vietnam

Course: SWR302 – Class: SE20A02 – Semester: SU26 – Group 3  
{vyho0509, tiendat1177sg, huykhanght17, tanphatho03,  
chuotphuc311005}@gmail.com

**Abstract.** Modern sports complexes face significant operational inefficiencies, including manual booking conflicts, financial deficits caused by user no-shows in recreational matches, and low conversion rates in premium equipment sales. This paper presents the requirements engineering and system specification for *Pro-Sport Complex Management*, a comprehensive web-based system engineered to digitalize and optimize sports facility operations, specifically tailored for Badminton and Pickleball. The system introduces an automated real-time booking scheduler integrated with programmatic slot-locking mechanisms to prevent race conditions. Furthermore, we specify a novel escrow-based "Social Matching" (Cap-keo) functional requirements framework that mitigates financial risks of abandonment by dynamically distributing court expenses through a secure intermediary wallet topology. To bridge the gap between facility usage and retail, a "Try-Before-You-Buy" lifecycle model is defined to translate on-premise rental fees into e-commerce discount vouchers while tracking asset depreciation via quantitative utilization metrics. Built upon a decoupled ReactJS frontend and a strongly transactional Java Web MVC/Hibernate backend with Microsoft SQL Server, this paper presents the formal software specification, actor interactions, and functional dependencies required to realize the platform.

**Keywords:** Requirements Engineering · Use Case Specification · Escrow Protocol · Race Condition · Try-Before-You-Buy.

## 1 Introduction

In the contemporary urban landscape, participation in amateur sports—particularly high-intensity racket sports such as Badminton and Pickleball—has experienced exponential growth. This surge has placed a heavy administrative burden on traditional sports complex operators. Most physical venues continue to rely on manual scheduling methods, such as logbooks, fragmented instant messaging apps, or unstructured spreadsheets. This historical dependency frequently leads to severe operational friction, including booking conflicts, multi-channel scheduling

overlaps, and substantial administrative labor costs. From the customer's perspective, the process of finding available slots during peak hours remains opaque and inefficient.

Beyond scheduling logistics, two systemic problems disrupt the amateur sports community: financial vulnerability in casual match organization and low retail conversion for specialized sports equipment. In grass-roots sports circles, players frequently organize matches with acquaintances or strangers (locally known as "Cap-keo") to share court costs. However, organizers regularly incur full financial liability due to last-minute cancellations or "no-show" participants, as there is no formal mechanism to enforce financial accountability. Concurrently, while players desire to test professional-grade rackets or equipment prior to purchasing, existing pro-shops lack transparent asset depreciation workflows. Consequently, they impose prohibitively high security deposits, resulting in low retail conversion rates and untapped immediate consumer demand.

To address these multi-faceted challenges, this study designs and specifies the functional domain for *Pro-Sport Complex Management*, a highly secure, transactional, and integrated software ecosystem. The system unifies three core domains: real-time smart booking, an escrow-based social matchmaking platform, and an on-premise rental subsystem linked directly to an e-commerce workflow.

The primary software requirements contributions analyzed in this paper are organized as follows:

- We define a comprehensive Actor-Use Case matrix establishing multi-tiered role access boundaries (Customer, Staff, Admin) with integrated E-KYC identification requirements.
- We model the real-time smart booking domain requirements featuring parameterized slot-locking behaviors that prevent concurrent transaction conflicts.
- We formalize a programmatic escrow-based financial requirement framework for amateur sports matching, which dynamically holds and redistributes court fees to eliminate user abandonment risks.

## 2 Related Work

The application of Information Technology in sports facility scheduling and requirements modeling has been a rich area of research over the past decade. Early iterations of sports management systems focused primarily on basic reservation workflows and static scheduling algorithms.

### 2.1 Sports Facility Reservation and Concurrency Control

Several authors have explored the functional specification of court reservations. Previous works utilized standard web architectures to replace manual ledger

books. However, as noted by Smith et al. [1], early system specifications frequently overlooked the "race condition" problem during peak hours, where two distinct actors concurrently attempt to reserve the same time slot on the same court. Modern consensus dictates that requirements engineers must explicitly specify data integrity constraints at the persistence tier, leveraging Object-Relational Mapping (ORM) pessimistic locking or strict isolation levels within the software requirements specification (SRS).

## 2.2 Collaborative Social Platforms and Financial Safeguards

The rise of the sharing economy has influenced casual sports scheduling. Mobile applications have emerged to connect local players based on geographical proximity and skill levels. However, these system requirements operate predominantly as communication intermediaries, leaving the actual collection of fees to peer-to-peer cash transfers. This structural limitation exposes match organizers to high financial risks. While escrow protocols are heavily specified in large-scale international trade platforms or digital freelancing networks [2], their behavioral requirements within micro-transactional, localized sports matching frameworks remain largely unexamined.

## 2.3 Product-Service Systems (PSS) and Try-Before-You-Buy Models

The "Try-Before-You-Buy" (TBYB) retail paradigm has gained substantial traction in global e-commerce, particularly for fashion and premium electronics. In the context of sports gear, requirements models typically separate rental fleets from retail stock, creating disconnected data silos. Researchers in software engineering emphasize that tracking asset lifecycle via a dedicated counter is vital to predicting asset degradation [3]. Our work extends this requirement domain by establishing an automated bridge between the physical rental state (on-premise POS) and digital retail platforms via dynamic voucher emission.

---

Method To construct a highly maintainable and technically rigorous system, the requirements schema for the Pro-Sport Complex Management platform is mapped using strict structural and behavioral software engineering dimensions.

## 2.4 Actor Analysis and System Boundaries

The functional ecosystem of the platform is bounded by three primary human actors and two external automated subsystems. Defining these clear boundaries prevents scope creep during implementation:

1. **Customer (Player):** The primary end-user who initiates real-time slot bookings, uploads E-KYC documentation (Student ID/CCCD) for verification, posts "Cap-keo" matchmaking requests, deposits escrow funds, and completes e-commerce acquisitions.

2. **Staff (On-premise Operator):** The operational actor responsible for validating physical E-KYC credentials, managing the Point-of-Sale (POS) physical rental workflow, and conducting the objective "Condition Check" upon asset return.
3. **Admin (Manager/Owner):** The high-privilege actor who monitors consolidated financial dashboards, defines system-wide configuration variables (such as court rates and conversion coefficients), and manages multi-warehouse allocations.
4. **External Payment Gateway (VNPay/MoMo APIs):** An external system actor that handles financial processing asynchronously and returns transactional success/failure webhooks to the core system.

## 2.5 Functional Requirements Specification (Use Case Modeling)

The system core is organized into three macro functional requirement modules, each governed by specific transactional boundaries and logical constraints.

**1. Smart Booking Requirements and Concurrency Control** The system must expose a real-time interactive schedule matrix. To eliminate concurrent transaction overwrite anomalies (Race Conditions), the requirement specifications mandate a multi-state transactional machine:

$$\text{Slot\_State}(C_i, T) = \begin{cases} \text{Available,} & \text{if Unreserved} \\ \text{Locked,} & \text{if Payment\_In\_Progress} \\ \text{Reserved,} & \text{if Transaction\_Success} \end{cases} \quad (1)$$

When a Customer triggers the **Select Slot** use case, the system transitions the state to Locked. This state change triggers an immediate **PESSIMISTIC\_WRITE** lock within the underlying database layer. The system sets an immutable non-functional timeout constraint of 900 seconds (15 minutes). If the External Payment Gateway does not broadcast a successful payment verification webhook within this temporal boundary, an automated background daemon releases the lock, returning the slot to Available.

**2. Escrow-Based Social Matching ("Cap-keo") Protocol** The matchmaking module functions as a programmatic escrow mediator to protect users against financial desertion ("bùng kèo"). The functional specification flows through three sequential milestones:

1. **Initialization Phase:** The Host Customer creates a match event specifying total court expenses ( $P_{total}$ ) and participant capacity ( $N_{capacity}$ ). The system dynamically computes the mandatory participant pledge ( $P_{individual}$ ):

$$P_{individual} = \frac{P_{total}}{N_{capacity} + 1} \quad (2)$$

2. **Escrow Phase:** To execute the **Join Match** use case, Guest Customers must pre-authorize and deposit  $P_{individual}$  into the system’s designated intermediary escrow vault ledger.
3. **Disbursement Phase:** Upon match completion, the Staff executes the **Verify Attendance** use case. The backend Model layer evaluates the attendance array and applies the following conditional forks:
  - **Condition A (Present):** The player’s held  $P_{individual}$  is successfully routed to the Host’s system wallet balance.
  - **Condition B (No-Show):** The absent player’s held  $P_{individual}$  is permanently forfeited and immediately disbursed to the Host as a liquidated penalty asset.

**Fig. 1.** Core Use Case relationships for the Smart Booking and Social Matching domains.

**3. Try-Before-You-Buy (TBYB) Inventory Lifecycle** The inventory requirement specification mandates strict logical separation between pristine stock (*New Inventory*) and physical trial assets (*Rental Inventory*). Every physical trial asset maps to an operational tracking variable,  $Rent\_Count$ .

When a Customer returns a rented tool, the Staff triggers the **Process Return** use case, prompting an explicit input requirement for the physical assessment (Condition Check): Normal, Scratched, or Damaged. An assessment of Damaged forces the backend to append an automated financial surcharge ( $Fee_{surcharge}$ ).

Conversely, a successful, non-damaged return automatically fires an asynchronous event to the voucher emission micro-service, computing a retail incentive:

$$Voucher\_Value = Rental\_Fee_{paid} \times \alpha \quad (3)$$

Where  $\alpha \in [0, 1]$  represents the system-wide management-configured retail conversion coefficient. The requirements enforce two non-functional constraints on the emitted entity: it must expire precisely 24 hours post-generation, and it must contain a relational constraint binding it exclusively to the purchasing Customer account UUID to eliminate fraudulent voucher trading.

## 2.6 System Architecture and Data Modeling

To fulfill the specified functional requirements and support high concurrency, the platform adopts a decoupled multi-tier architecture. The Client Tier (Frontend) is engineered using ReactJS and Tailwind CSS, providing a responsive, state-driven user experience. The Business Tier (Backend) operates on Java Web Servlets and Hibernate ORM, acting as a robust transactional engine.

Data persistence is governed by an Entity-Relationship (ER) schema deployed on Microsoft SQL Server. Crucial entities include `Account`, `Court_Slot`, `Match_Escrow`, and `Rental_Asset`. To guarantee data integrity during the "Cap-keo" disbursement phase, the `Match_Escrow` entity strictly enforces ACID properties, ensuring that partial database updates cannot cause financial discrepancies between the Host and Guest wallets.

## 2.7 User Interface (UI) Implementation Strategy

The frontend methodology prioritizes frictionless navigation, cognitive clarity, and real-time feedback, aligning with modern Human-Computer Interaction (HCI) standards. Utilizing the Shaden UI component library, the system translates complex data into intuitive visual paradigms:

- **Dynamic Booking Matrix:** The Smart Booking interface deploys a temporal grid system where court slot availability is distinctly color-coded (e.g., Green for Available, Yellow for Locked/Pending Payment, Red for Reserved).
- **Escrow Matchmaking Cards:** The "Cap-keo" social matching feed utilizes a card-based layout. Each component explicitly displays critical match metrics (Total Cost  $P_{total}$ , Required Skill Level, Available Slots) alongside a prominent call-to-action (CTA) for escrow deposit authorization.
- **Reactive State Management:** Leveraging React Hooks, the frontend architecture ensures that backend payload updates regarding court status or inventory condition checks instantly trigger isolated re-renders of the Virtual DOM, eliminating the need for full-page reloads and maintaining a seamless user flow.

## Acknowledgements

The authors express their sincere gratitude to Ms. Tran Thi To Tam (TamTTT14), Lecturer at the Department of Software Engineering, FPT University, Da Nang, for her valuable guidance, continuous academic support, and constructive feedback throughout the requirements analysis phase of this project.

## References

## References

1. Smith, J., Johnson, M., Davis, R.: Concurrency control and transactional integrity in modern facility scheduling systems. *Journal of Software Engineering and Applications* **14**(5), 234–248 (2021).

2. Johnson, A., Lee, S.: Micro-escrow protocols for peer-to-peer sharing economies. In: International Conference on Digital Trust and Blockchain Technology, pp. 112–126. Springer, Cham (2023).
3. Lee, K., Park, T.: Predictive maintenance and depreciation modeling for sports equipment leasing networks. *International Journal of Production Economics* **245**, 108–119 (2022).
4. Gamma, E., Helm, R., Johnson, R., Vlissides, J.: *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Reading (1994).
5. Bauer, C., King, G., Gregory, G.: *Java Persistence with Hibernate*, 2nd edn. Manning Publications, New York (2016).
6. Vietnam Payment Solution Joint Stock Company: Technical integration specifications for asynchronous financial webhooks, v4.1.2. Internal Documentation (2025).